



电子科技大学

University of Electronic Science and Technology of China

信息与软件工程学院

SCHOOL OF INFORMATION AND SOFTWARE ENGINEERING

W1D1: Linux开发环境

Linux下Web Server的设计

讲授内容

- 训练目的
- 训练原理
- 训练内容
- Webserver V0.1版
 - 基本理解Webserver功能
 - 搭建webserver工程结构



Webserver V0.1版

建立Webserver工程目录及读写功能

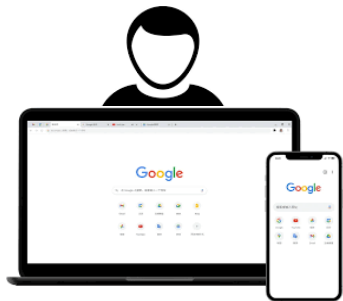


电子科技大学
University of Electronic Science and Technology of China

信息与软件工程学院
SCHOOL OF INFORMATION AND SOFTWARE ENGINEERING

Webserver简单理解

- 用户访问网站的过程



用户打开浏览器,
输入: www.baidu.com,
回车

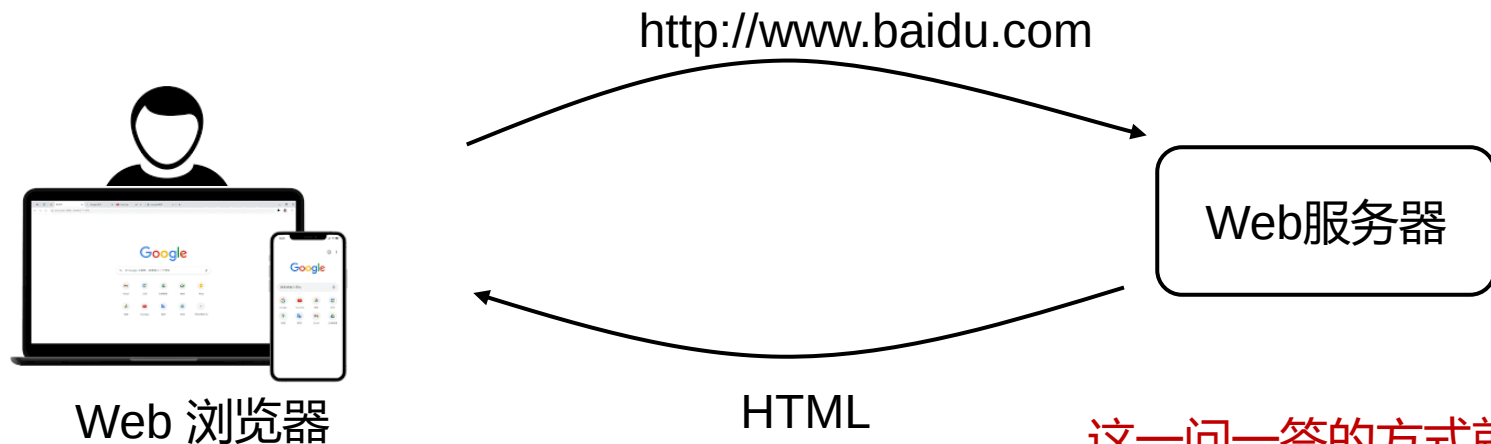


电子科技大学
University of Electronic Science and Technology of China

信息与软件工程学院
SCHOOL OF INFORMATION AND SOFTWARE ENGINEERING

Webserver简单理解

- 用户访问网站的过程



这一问一答的方式就被
成为：请求/响应模式

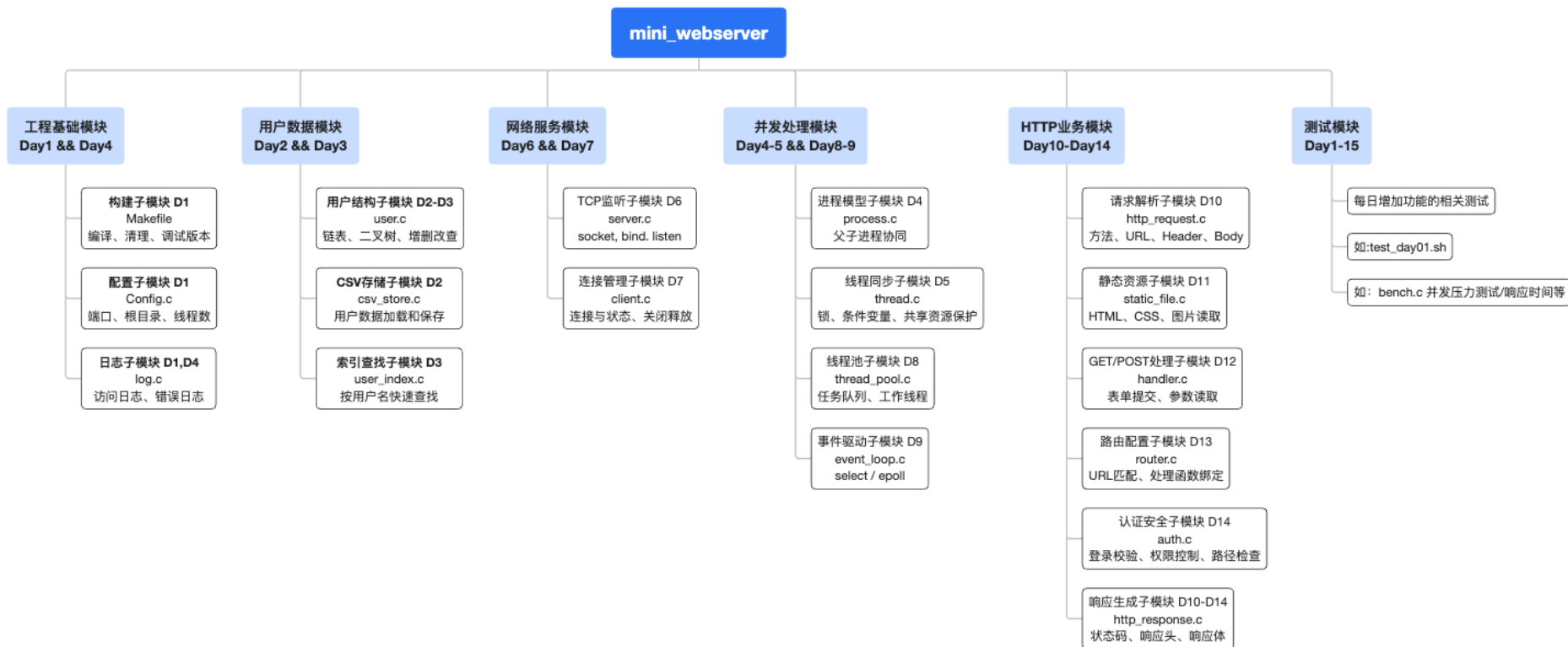
1. 用户向Web服务器发起一个HTTP请求
2. Web服务器向用户返回一个HTML页面
3. 用户的浏览器会将页面渲染成用户最终看到的效果



- 第1天：工程骨架、构建、配置、日志
- 第2天：用户数据结构和CSV读写
- 第3天：用户索引和查找
- 第4天：进程模型和日志子进程
- 第5天：线程和共享资源保护
- 第6天：TCP服务器
- 第7天：多客户端连接
- 第8天：线程池
- 第9天：select/epoll
- 第10天：HTTP请求解析
- 第11天：静态资源
- 第12天：GET/POST
- 第13天：路由和配置
- 第14天：认证和安全
- 第15天：压测、演示、答辩



Webserver的功能模块图



搭建V0.1版的工程目录 mini_webserver

```
mini_webserver/  
  include/  
    config.h  
    log.h  
    http_response.h  
  src/  
    main.c  
    config.c  
    log.c  
    http_response.c  
  tests/  
    test_day01.sh  
  conf/  
    server.conf  
  www/  
    index.html  
  logs/  
  docs/  
    debug-log.txt  
  Makefile
```

工程结构



搭建V0.1版的工程目录 mini_webserver

```
#ifndef CONFIG_H
#define CONFIG_H
typedef struct {
    char server_name[64];
    char host[64];
    int port;
    char root[128];
    char log_path[128];
} server_config_t;
```

```
int load_config(const char *path, server_config_t *config);
void print_config(const server_config_t *config);
#endif
```

config.h

支持load_config 能从 conf/server.conf 读取配置



搭建V0.1版的工程目录 mini_webserver

```
#include "config.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
static void remove_newline(char *text) { 【todo】 }
int load_config(const char *path, server_config_t *config) {
    FILE *fp = fopen(path, "r");
    char line[256];
    if (fp == NULL || config == NULL) {return -1; }
    while (fgets(line, sizeof(line), fp) != NULL) {
        char *key;
        char *value;
        remove_newline(line);
        key = strtok(line, "=");
        value = strtok(NULL, "=");
        if (strcmp(key, "server_name") == 0) {
            strncpy(config->server_name, value, sizeof(config->server_name) - 1);
        } else if (strcmp(key, "host") == 0) { 【todo】 }
        } else if (strcmp(key, "port") == 0) { 【todo】 }
        } else if (strcmp(key, "root") == 0) { 【todo】 }
        } else if (strcmp(key, "log") == 0) { 【todo】 } }
    fclose(fp);
    return 0;}
```

config.c

```
void print_config(const server_config_t *config) {
    printf("server_name=%s\n", config->server_name);
    printf("host=%s\n", config->host);
    printf("port=%d\n", config->port);
    printf("root=%s\n", config->root);
    printf("log=%s\n", config->log_path);
}
```



搭建V0.1版的工程目录 mini_webserver

```
#ifndef LOG_H
#define LOG_H

int log_init(const char *path);
void log_info(const char *message);
void log_error(const char *message);
void log_close(void);

#endif
```

log.h

程序运行后能生成 logs/server.log



搭建V0.1版的工程目录 mini_webserver

```
#include "log.h"
#include <stdio.h>
static FILE *log_file = NULL;
int log_init(const char *path) {
    log_file = fopen(path, "a");
    if (log_file == NULL) {
        return -1;
    }
    return 0;
}
void log_info(const char *message) {
    if (log_file != NULL) {
        fprintf(log_file, "[INFO] %s\n", message);
        fflush(log_file);
    }
}
void log_error(const char *message) {
    if (log_file != NULL) {
        fprintf(log_file, "[ERROR] %s\n", message);
        fflush(log_file);
    }
}
void log_close(void) {
    if (log_file != NULL) {
        fclose(log_file);
        log_file = NULL;
    }
}
```

log.c



搭建V0.1版的工程目录 mini_webserver

```
#ifndef HTTP_RESPONSE_H
#define HTTP_RESPONSE_H

int build_hello_response(char *buffer, int size);

#endif
```

response.h

用于生成合法 HTTP 响应文本，包含状态行、响应头、空行和正文



搭建V0.1版的工程目录 mini_webserver

```
#include "http_response.h"
#include <stdio.h>
int build_hello_response(char *buffer, int size) {
    const char *body = "Hello, Web!\n";
    int written;
    if (buffer == NULL || size < 0) {
        return -1;
    }
    written = snprintf(
        buffer,
        size,
        "HTTP/1.1 200 OK\r\n"
        "Content-Type: text/html\r\n"
        "Content-Length: 12\r\n"
        "\r\n"
        "%s",
        body
    );
    if (written < 0) {
        return -1;
    }
    return 0;}

```

response.c



搭建V0.1版的工程目录 mini webserver

```
#include "config.h"
#include "http_response.h"
#include "log.h"
#include <stdio.h>
#include <string.h>
```

main.c

```
int main(int argc, char *argv[]) {
    server_config_t config;
    char response[64];
    if (argc != 2) {
        printf("usage: %s conf/server.conf\n", argv[0]);
        return 1;
    }
    memset(&config, 0, sizeof(config));
    if (load_config(argv[1], &config) != 0) {
        printf("failed to load config\n");
        return 1;
    }
    log_init(config.log_path);
    log_info("server config loaded");
    log_info("document root loaded");
    print_config(&config);
    if (build_hello_response(response, sizeof(response)) < 0) {
        log_error("failed to build response");
        log_close();
        return 1;
    }
    printf("%s", response);
    log_close();
    return 0;
}
```

搭建V0.1版的工程目录 mini_webserver

```
server_name=MiniWeb  
host=127.0.0.1
```

```
port = 8080
```

```
root=www
```

```
log=logs/server.log
```

conf/server.conf

```
<!doctype html>
```

```
<html>
```

```
<head>
```

```
  <meta charset="utf-8">
```

```
  <title>MiniWeb</title>
```

```
</head>
```

```
<body>
```

```
  <h1>Hello, MiniWeb</h1>
```

```
</body>
```

```
</html>
```

www/index.html



搭建V0.1版的工程目录 mini_webserver

功能实现后，要考虑对每次实现的功能皆进行测试

```
tests/  
    test_day01.sh
```

所有测试文件都放到tests文件夹下。

第一天的测试可以考虑使用批处理文件(.sh)来批量完成



搭建V0.1版的工程目录 mini_webserver

```
#!/usr/bin/env bash
set -e
```

```
rm -f logs/server.log
rm -f day01_output.txt
rm -f bad_log_output.txt
rm -f conf/bad_log.conf
```

```
make clean
make
```

```
./mini_web_server conf/server.conf > day01_output.txt
```

```
grep -F "server_name=MiniWeb" day01_output.txt
grep -F "host=127.0.0.1" day01_output.txt
grep -F "port=8080" day01_output.txt
grep -F "root=www" day01_output.txt
grep -F "HTTP/1.1 200 OK" day01_output.txt
grep -F "Content-Type: text/html" day01_output.txt
grep -F "Hello, Web!" day01_output.txt
```

```
test -f logs/server.log
grep -F "[INFO] server config loaded" logs/server.log
```

```
cat > conf/bad_log.conf <<'CONFIG'
server_name=BadLog
host=127.0.0.1
port=8080
root=www
log=logs/missing/server.log
CONFIG
```

```
set +e
./mini_web_server conf/bad_log.conf > bad_log_output.txt 2>&1
status=$?
set -e
```

```
test "$status" -ne 0
grep -F "failed to open log file" bad_log_output.txt
```

```
echo "DAY01 PASS"
```

Test_day01.sh



搭建V0.1版的工程目录 mini_webserver

- 运行
- make run
- 测试
- make test
- 清理
- make clean



以上工程代码预设了一些问题

- 编译不通过
- 修复后程序运行可能会崩溃等
- 测试内容异常



得到每个模块
的正常输出



搭建V0.1版的工程目录 mini_webserver

简单记录出现的问题

#使用的命令

#报错内容

#

例如：

`gdb ./mini_web_server`

`run conf/server.conf`

`debug-log.txt`



搭建V0.1版的工程目录 mini_web_server

可看到正常输出

```
server_name=MiniWeb  
host=127.0.0.1  
port=8080  
root=www  
log=logs/server.log  
HTTP/1.1 200 OK  
Content-Type: text/html  
Content-Length: 12  
Hello, Web!
```

logs/server.log 内容输出为

```
[INFO] server config loaded  
[INFO] document root loaded
```

此时如果没有正确配置Log目录应该出现:

```
failed to open log file
```

DAY01 PASS

